

Memory Circuits & Systems

Exercise 5

Ramulator2.0: DRAM Controller Simulator

Professor Po-Tsang Huang

Institute of Electronics

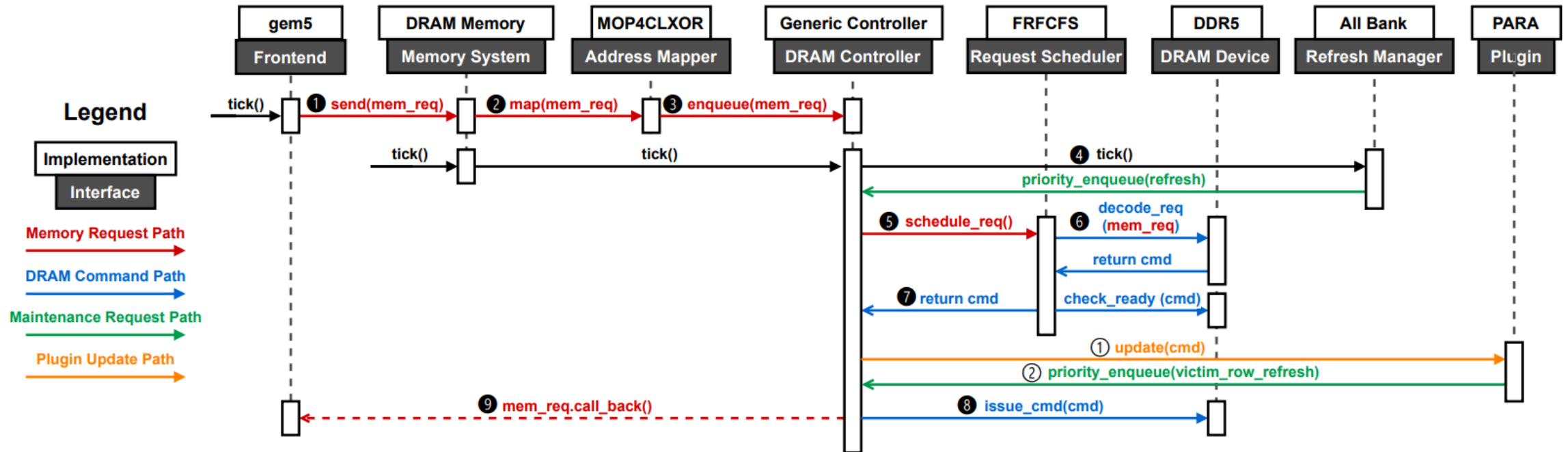
National Yang Ming Chiao Tung University



What is Ramulator2.0

- Cycle-level DRAM controller simulator
- Ramulator2.0 Can Model
 - ◆ Memory requests from traces
 - ◆ DRAM hierarchy
 - ◆ Memory controller behavior
 - ◆ DRAM timing constraints
 - ◆ Row buffer behavior
 - ◆ Performance statistics

Ramulator2.0 Simulation Flow



Setup

- `tar -xvf /RAID2/COURSE/2026_Spring/es26mcs/es26mcsTA05/MCS_EX5.tar`
- `cd ramulator2`
- `source .cshrc` (Ramulator2.0 needs cmake & new version of g++)

```
20:43 es26mcsTA04@ee31[~/MCS_EX5_YMM/final/ramulator2]$ source .cshrc  
In MCS Ramulator HW environment, do not type the keywords yes', 'layout' or 'coding'
```

- Run `./00_setup.tcl`

```
1  mkdir ./build  
2  mkdir ./traces  
3  mkdir ./traces_log  
4  mkdir ./cmd_records  
5  mkdir ./returned_trace  
6  mkdir ./sent_request_trace  
7  cp run_ramulator ./build/01_run_ramulator
```

- Note: the `./09_clean_up.tcl` will clean all the log file and compile file, **please use it carefully**

Prepare file

- `cd ./scripts` to gen the trace file
 - ◆ `python3 generate_trace.py --pattern both --st-count 256 --ld-count 256 --addr-space 1048576 --out-prefix trace`

- Trace file

```
trace_sequential.trace
ST 0x0 0 0
ST 0x80 10 0
ST 0x100 0 0
ST 0x180 0 0
ST 0x200 0 0
ST 0x280 0 0
ST 0x300 0 0
ST 0x380 0 0
ST 0x400 0 0
ST 0x480 0 0
ST 0x500 0 0
ST 0x580 0 0
ST 0x600 0 0
ST 0x680 0 0
ST 0x700 0 0
ST 0x780 0 0
```

- Config file

```
config.yaml
Frontend:
  impl: LoadStoreStallTrace
  clock_ratio: 3
  num_expected_insts: 512
  returned_trace_path: ../returned_trace/
  trace_served_file_path: ../sent_request_trace/
  bandwidth_sample_time_interval: 100
  debug: false
  traces:
    - ../traces/trace_sequential.trace

Translation:
  impl: NoTranslation
  max_addr: 33554432
```

← You need to change the trace file path right here

Compile

- cd build
- cmake ..

```
20:43 es26mcsTA04@ee31[~/MCS_EX5_YMM/final/ramulator2/build]% cmake ..
-- The CXX compiler identification is GNU 13.2.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /RAID2/COURSE/2026_Spring/es26mcs/es26mcsTA05/share/local_tools/gcc-13.2.0/bin/g++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
Configuring Ramulator ...
Configuring yaml-cpp...
CMake Deprecation Warning at ext/yaml-cpp/CMakeLists.txt:2 (cmake_minimum_required):
Compatibility with CMake < 3.5 will be removed from a future version of
CMake.

Update the VERSION argument <min> value or use a ...<max> suffix to tell
CMake that the project does not need compatibility with older versions.

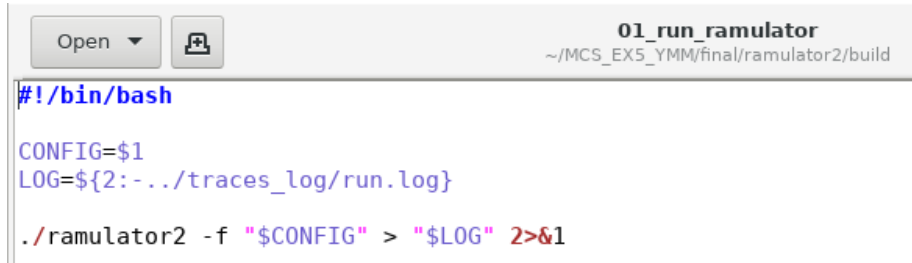
Done configuring yaml-cpp.
Configuring spdlog...
-- Build spdlog: 1.11.0
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Failed
-- Looking for pthread_create in pthreads
-- Looking for pthread_create in pthreads - not found
-- Looking for pthread_create in pthread
-- Looking for pthread_create in pthread - found
-- Found Threads: TRUE
-- Build type: Release
Done configuring spdlog.
Configuring argparse...
Done configuring argparse.
-- Configuring done (20.8s)
-- Generating done (2.3s)
-- Build files have been written to: /RAID2/COURSE/2026_Spring/es26mcs/es26mcsTA04/MCS_EX5_YMM/final/ramulator2/build
```

- make -j8

```
20:43 es26mcsTA04@ee31[~/MCS_EX5_YMM/final/ramulator2/build]% make -j8
[ 8%] Building CXX object _deps/spdlog-build/CMakeFiles/spdlog.dir/src/spdlog.cpp.o
[ 8%] Building CXX object src/translation/CMakeFiles/ramulator-translation.dir/impl/no_translation.cpp.o
[ 8%] Building CXX object src/base/CMakeFiles/ramulator-base.dir/factory.cpp.o
[ 8%] Building CXX object src/test/CMakeFiles/ramulator-test.dir/test_impl.cpp.o
[ 8%] Building CXX object src/memory_system/CMakeFiles/ramulator-memorysystem.dir/impl/Global_Controller.cpp.o
[ 96%] Building CXX object src/dram_controller/CMakeFiles/ramulator-controller.dir/impl/plugin/prac/prac.cpp.o
[ 96%] Built target ramulator-controller
[ 97%] Linking CXX shared library /RAID2/COURSE/2026_Spring/es26mcs/es26mcsTA04/MCS_EX5_YMM/final/ramulator2/libramulator.so
[ 97%] Built target ramulator
[ 98%] Building CXX object CMakeFiles/ramulator-exe.dir/src/main.cpp.o
[100%] Linking CXX executable ramulator2
[100%] Built target ramulator-exe
```

Run Ramulator2.0

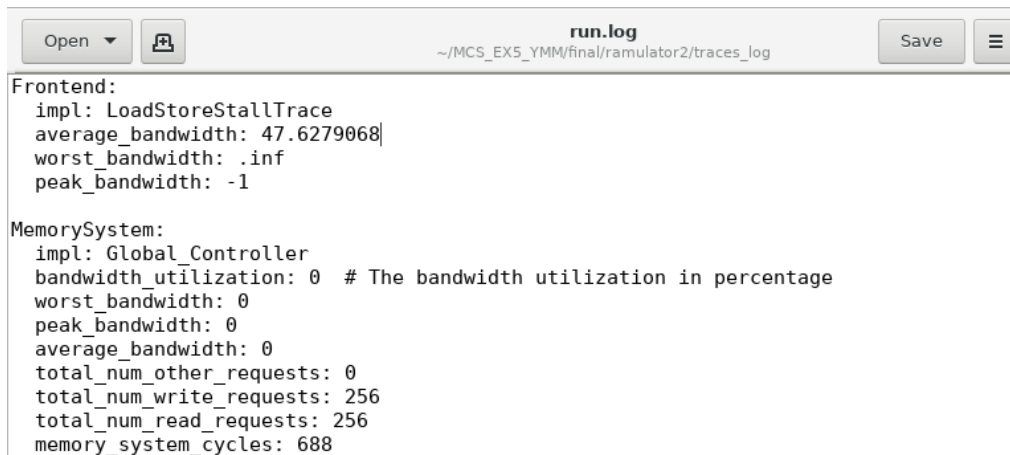
- `./01_run_ramulator ../config_files/config.yaml`



```
#!/bin/bash
CONFIG=$1
LOG=${2:-../traces_log/run.log}
./ramulator2 -f "$CONFIG" > "$LOG" 2>&1
```

```
22:04 es26mcsTA04@ee31[~/MCS_EX5_YMM/final/ramulator2/build]$ ./01_run_ramulator ../config_files/config.yaml
```

- The result will store in `../traces_log/run.log`



```
Frontend:
impl: LoadStoreStallTrace
average_bandwidth: 47.6279068
worst_bandwidth: .inf
peak_bandwidth: -1

MemorySystem:
impl: Global_Controller
bandwidth_utilization: 0 # The bandwidth utilization in percentage
worst_bandwidth: 0
peak_bandwidth: 0
average_bandwidth: 0
total_num_other_requests: 0
total_num_write_requests: 256
total_num_read_requests: 256
memory_system_cycles: 688
```

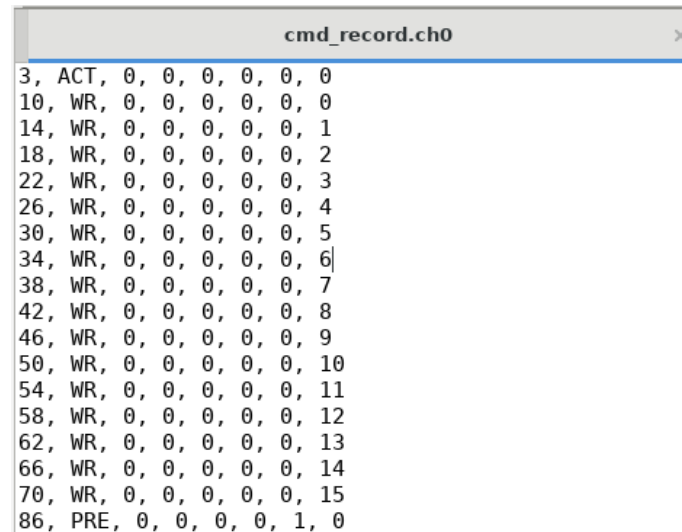
TraceRecorder Plugin

- Check DRAM timing behavior
- Observe row hit / row conflict
- Understand row policy behavior
- Explain scheduling decisions

```
- ControllerPlugin:  
  impl: TraceRecorder  
  path: ../cmd_records/cmd_records.trace
```

In src/dram/impl/DDR4.cpp

```
inline static constexpr ImplDef m_levels = {  
  "channel", "rank", "bankgroup", "bank", "row", "column",  
};
```



cmd_record.ch0									
3	ACT	0	0	0	0	0	0	0	0
10	WR	0	0	0	0	0	0	0	0
14	WR	0	0	0	0	0	0	1	
18	WR	0	0	0	0	0	0	2	
22	WR	0	0	0	0	0	0	3	
26	WR	0	0	0	0	0	0	4	
30	WR	0	0	0	0	0	0	5	
34	WR	0	0	0	0	0	0	6	
38	WR	0	0	0	0	0	0	7	
42	WR	0	0	0	0	0	0	8	
46	WR	0	0	0	0	0	0	9	
50	WR	0	0	0	0	0	0	10	
54	WR	0	0	0	0	0	0	11	
58	WR	0	0	0	0	0	0	12	
62	WR	0	0	0	0	0	0	13	
66	WR	0	0	0	0	0	0	14	
70	WR	0	0	0	0	0	0	15	
86	PRE	0	0	0	0	0	1	0	

1.Address mapping (50%)

- DRAM Architecture in this lab
 - ◆ 4 channels, 1 bank per channel, 65536 rows per bank, 16 columns per row
- Timing Constraint determine when a DRAM command can be issued
 - ◆ tRCD: ACT → READ/WRITE delay
 - ◆ tRP: PRE → Next ACT delay
 - ◆ tCL: Time from READ command to data available
 - ◆ tRAS: Minimum row active time
 - ◆ tRC: Minimum time between two ACTs to the same bank
 - ◆ tRFC: Time required to complete a refresh operation
- DRAM configuration
 - ◆ File: src/dram/impl/DDR4.cpp

```
I0) // name rate nBL nCL nRCD nRP nRAS nRC nWR nRTP nCWL(TSV as
nCCDS nCCDL nRRDS nRRDL nWTRS nWTRL nFAW nRFC nREFI nCS, tCK_ps
{"DDR4_3DDR4_1024", {2000, 2, 8, -1, -1, -1, 2, 1000}},|
5, 4, 4, -1, -1, 8, 8, -1, -1, -1, 2, 1000}},|
```

1.Address mapping (50%)

- Q1-1: Analysis with Given Address Mapping
 - ◆ Run **sequential** and **random** traces using the provided address mapping.
 - ◆ Analyze and explain
 - The issued DRAM command sequence (ACT / READ / PRE)
 - Why each command is issued
 - Which timing constraints delay the next command

1.Address mapping (50%)

- Q1-2: Design a new address mapping scheme to improve memory performance.
 - ◆ Modify `src/addr_mapper/impl/linear_mappers.cpp` to change the address mapping scheme.
 - Hint: modify the different order for Ch/Ra/Ba/Row/Col
 - **You need to compile again!!**
 - ◆ Analysis how to increase the effective memory access bandwidth.

```
class New_mapping final : public LinearMapperBase, public Implementation {
    RAMULATOR_REGISTER_IMPLEMENTATION(IAddrMapper, New_mapping, "New_mapping", "Applies a New_mapping mapping to the address.");
public:
    void init() override { };

    void setup(IFrontEnd* frontend, IMemorySystem* memory_system) override {
        LinearMapperBase::setup(frontend, memory_system);
    }

    void apply(Request& req) override {
        req.addr_vec.resize(m_num_levels, -1);
        Addr_t addr = req.addr >> m_tx_offset;

        // TO DO HERE
    }
};
```

```
inline static constexpr ImplDef m_levels = {
    "channel", "rank", "bankgroup", "bank", "row", "column",
};
```

2.Row-policy (50%)

■ Row-Buffer Behavior

- ◆ A DRAM bank can keep only one row active at a time
- ◆ Accessing the currently open row results in a Row Hit
- ◆ Accessing a different row causes a Row Conflict, requiring PRECHARGE + ACTIVATE operations.

■ Open-row policy

- ◆ Keeps the active row open after a memory access
- ◆ Increases the probability of future **row hits**

■ Close-row policy

- ◆ Closes the active row after a certain number of accesses
- ◆ Reduces row conflicts for workloads with low row locality or random accesses

■ File Path: src/dram_controller/impl/rowpolicy/basic_rowpolicies.cpp

■ Cap: Maximum number of column accesses allowed to the same row in Close row policy

```
RowPolicy:  
  impl: OpenRowPolicy|  
  cap: 4
```

```
RowPolicy:  
  impl: ClosedRowPolicy  
  cap: 4
```

2.Row-policy (50%)

- Q2-1: Row-Policy Analysis
 - ◆ Run the sequential and random traces and compare the behavior of open-row and close-row policies.
 - Any address mapping scheme may be used. Please clearly state the mapping used in your experiments.
 - ◆ Analyze and report
 - Memory bandwidth, Number of row hits, Number of row conflicts
 - DRAM command distribution (ACT / WRITE / READ / PRE)
 - ◆ Which traces type is suitable for open-row which for close-row

Report notice

- In your analysis please provide evidence to support your reason like log_file screenshot, code modify screenshot, etc.

Submission on E3 platform

- Please compress your **report & linear_mappers.cpp or another code you added in a single compressed file(.zip)** and upload this single file on E3 platform
 - ◆ **Report: Simulation result, comparison, analysis and the evidence picture.**
 - ◆ Naming rule: **Ex5_#ID.zip** (including report & code)
- Due date: 6/23 P.M 23:59